

MODULE 4.3: MODELING SEQUENCE AND TIME-SERIES DATA

Contents

Modeling Sequence and Time-Series Data	1
Introduction to Sequential / Temporal Data.....	2
Sequence Modeling Architectures.....	4
Feature Representation Learning	7
Feature Engineering.....	8
Explain the Three Techniques Used Under Supervised Feature Selection	8
Explain the Benefits of Using Feature Selection in Machine Learning.....	9

Modeling Sequence and Time-Series Data

- Modeling sequence and time-series data refers to building machine learning models that can **understand patterns over time** or **patterns across ordered data**.
- Unlike normal (i.i.d/ independent and identically distributed) datasets, sequence data has **order**, **context**, and **temporal dependence**.

What Counts as Sequence / Time-Series Data

Data Type	Examples
Time-Series Data	Stock prices, weather readings, ECG signals, sensor data
Sequential Data	Text sentences, speech audio, DNA sequences, video frames

Both share one property: **current value depends on previous values**.

Why Normal ML Models Don't Work

Traditional ML models treat each sample **independently**, but sequence data requires the model to **remember the past**. For example:

- Predicting tomorrow's temperature depends on earlier temperatures
- Predicting the next word depends on previous words

This need for **context** leads to special models.

How We Model Sequence / Time-Series Data

We use models that can store and update **memory** over time:

a) Recurrent Neural Networks (RNNs)

- Process input **step by step**
- Maintain a **hidden state** (memory)
- Good for basic sequence dependencies
- Example: sentiment analysis, simple forecasting

b) LSTM (Long Short-Term Memory)

- Designed to capture **long-term dependencies**
- Avoids vanishing gradient problem
- Example: text generation, speech models

c) GRU (Gated Recurrent Unit)

- Faster, simpler version of LSTM
- Example: chatbots, time-series signals

d) 1D CNN for Time-Series

- Extracts local temporal patterns (like peaks, spikes)
- Example: ECG classification

e) Transformers

- Current state-of-the-art
- Use **self-attention** to model long-range dependencies
- Example: GPT, BERT, translation, summarization

Goals in Sequence Modeling

- **Prediction:** Forecast the next value (e.g., stock price, temperature)
- **Classification:** Label sequences (e.g., detect anomaly in ECG)
- **Generation:** Produce new sequences (e.g., text generation)
- **Sequence-to-Sequence:** Convert one sequence to another (e.g., translation)

Challenges

- Long-term dependency
- Noisy sequences
- Irregular sampling
- Non-stationary data (changing patterns over time)

Introduction to Sequential / Temporal Data

- Sequential or temporal data is any data where the **order of observations matters**.
- Each data point depends on the **previous** data points, forming a sequence.
- In simple words: **The past affects the future.**

Types of Sequential Data

Type	Description	Examples
Time-Series Data	Data indexed by time; evenly or unevenly spaced.	Stock prices, weather readings, ECG signals
Ordered Sequences	Order matters but not tied to time.	Text sentences, DNA sequences, clickstreams
Spatiotemporal Data	Has both time and spatial components.	Video frames, traffic data

Key Characteristics

1. Temporal Ordering

- Time-series data always follows a strict chronological order.
- Each observation is tied to a specific timestamp, and the meaning of the data depends on this order.
- If you shuffle the sequence, the patterns, trends, and relationships break completely.

2. Temporal Dependence (Autocorrelation)

- The current value in a time-series is influenced by its previous values.
- Meaning: past behavior directly affects future behavior.
- For example, tomorrow's temperature often depends on today's temperature due to natural continuity.

3. Trend

- A time-series often shows a long-term upward or downward movement.
- Trends represent how the overall level of the data changes over months or years.
- Example: rising population, increasing sales over many years.

4. Seasonality

- Seasonality refers to short-term, regular, repeating patterns within fixed intervals.
- It could be daily, weekly, monthly, or yearly.
- Example: electricity usage increases every summer, or online shopping spikes every weekend.

5. Cyclic Patterns

- Cycles are long-term oscillations that occur irregularly, unlike seasonality.
- They depend on broader factors like economy, politics, or business conditions.
- Example: financial market boom and recession cycles.

6. Non-Stationarity

- The statistical properties of the series — mean, variance, covariance — may change over time.
- A non-stationary series behaves differently in different time periods, making prediction harder.
- Most real-world series need transformations (like differencing) to become stationary.

7. Noise / Random Fluctuations

- Time-series data often contains unexplained random variations caused by external factors.
- Noise does not follow any pattern but is superimposed on the underlying trend and seasonality.
- Example: sudden rain affecting daily sales.

8. Irregular Sampling

- Some time-series are collected at inconsistent or irregular time intervals.
- For example, heart-monitor data sampled when an event occurs, not at fixed seconds.
- This irregularity requires special handling in models.

9. Multivariate Relationships

- A time-series may be influenced by other related variables recorded over time.
- These additional variables help explain changes in the main series.
- Example: temperature depends on humidity, wind speed, and cloud cover.

10. Temporal Granularity

- The frequency at which data is recorded affects the patterns we observe.
- Hourly data shows short-term fluctuations; yearly data reveals long-term trends.
- Different granularities tell different stories of the same data.

Why Special Models Are Needed

- Normal ML models treat samples independently.
- But sequential data requires **memory**, i.e., the ability to use past information to understand or predict the present. This is why the following models are used for sequence learning:
 - **RNN**
 - **LSTM**
 - **GRU**
 - **1D CNN**
 - **Transformers**

Examples of Sequential / Temporal Data

- **Weather** readings over days
- **Stock prices** over months
- **Sensor data** from IoT devices
- **Speech** (sequence of audio frames)
- **Text** (sequence of words or characters)
- **Video** (sequence of frames)

Key Tasks in Sequential Modeling

Task	Description
Forecasting	Predict future values (weather, sales).
Classification	Assign labels (emotion detection in speech, ECG arrhythmia).
Sequence Generation	Generate new text, music, or signals.
Sequence-to-Sequence	Translation, captioning, summarization.

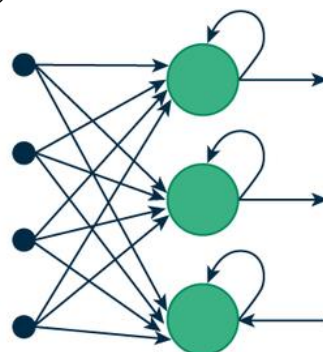
Challenges

- Long-term dependency
- Noise and irregular sampling
- Changes in data behaviour over time
- Vanishing/exploding gradients (in RNNs)

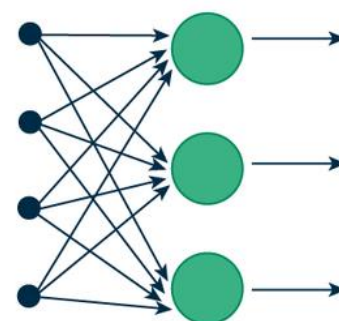
Sequence Modeling Architectures

1. Recurrent Neural Network (RNN)

- RNNs are neural networks designed to handle **sequential data**, where each output depends on both the **current input** and the **previous hidden state**.
- They introduce a form of **memory**, allowing the network to retain information from earlier time steps.
- The hidden state is updated at every step, enabling RNNs to model temporal patterns like text, speech, or sensor data.



(a) Recurrent Neural Network

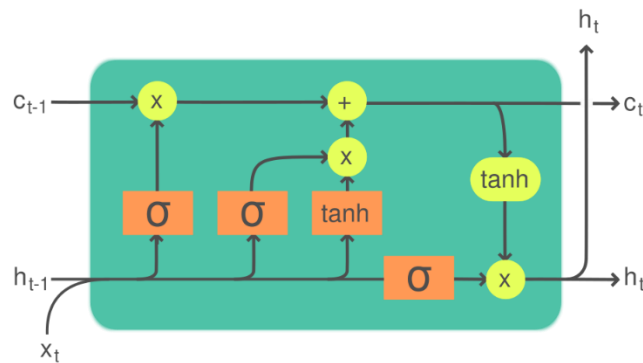


(b) Feed-Forward Neural Network

- However, basic RNNs struggle with **long-term dependencies** due to **vanishing and exploding gradients**, limiting their ability to remember distant past information.

2. Long Short-Term Memory (LSTM)

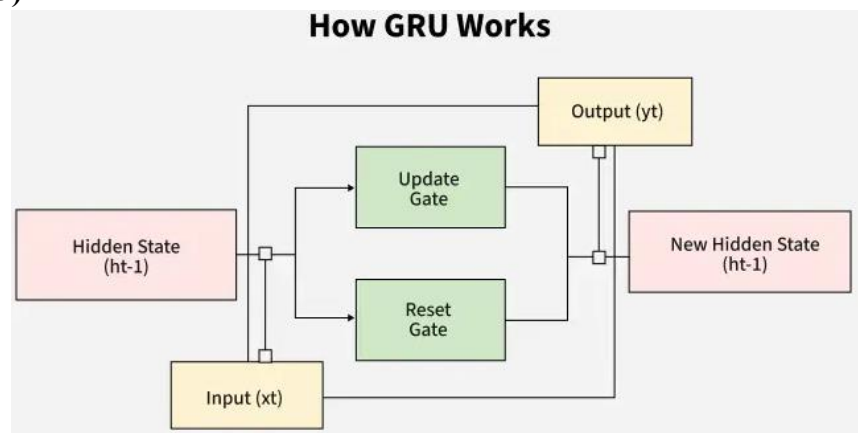
- LSTM is an improved RNN architecture designed to solve the vanishing gradient problem and handle **long-range dependencies** effectively.
- It uses a **cell state** (long-term memory) and three **gates**—input, forget, and output gates—to control the flow of information.
- These gates decide what to store, what to forget, and what to output at each time step, allowing the LSTM to remember information over long sequences.
- LSTMs are widely used in language modeling, speech recognition, machine translation, and time-series forecasting.



Legend: Layer Componentwise Copy Concatenate

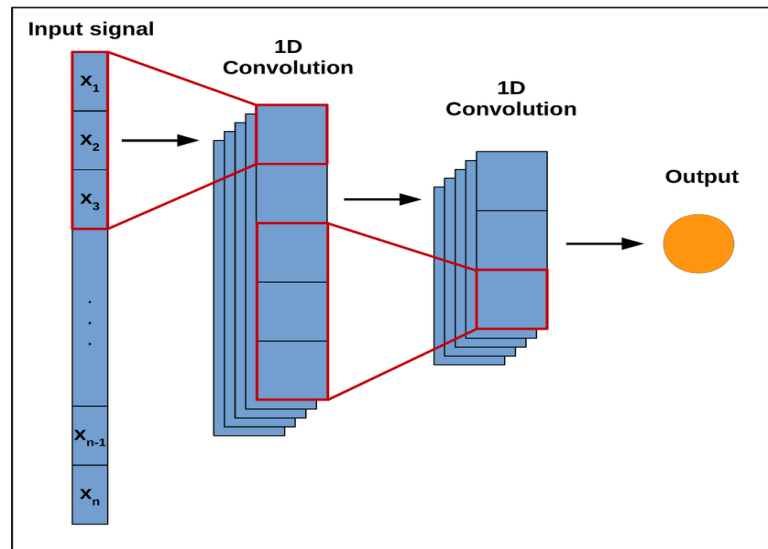
3. Gated Recurrent Unit (GRU)

- GRU is a simplified alternative to LSTM that uses only **two gates**—reset and update—while removing the separate cell state.
- It combines short-term and long-term memory into a single hidden state, making it **faster and easier to train** without losing performance.
- Despite its simpler structure, GRUs often match or outperform LSTMs on many sequence modeling tasks.
- They are popular in applications requiring efficient training, such as chatbots, speech systems, and real-time prediction.



4. 1D Convolutional Neural Network (1D CNN)

- A 1D CNN applies **convolution filters** across **temporal or sequential data**, extracting local patterns such as spikes, trends, or repeated structures.
- Instead of modeling memory explicitly like RNNs, 1D CNNs learn **local temporal dependencies** using sliding kernels.
- They are efficient, parallelizable, and often outperform RNNs on tasks with local structure, such as ECG classification, audio processing, sensor analysis, and sequence classification.
- They are widely used for time-series and NLP tasks when long-range dependencies are not required.



5. Transformers

- Transformers are state-of-the-art models that rely on **self-attention** rather than recurrence or convolution.
- Self-attention allows the model to directly look at **all positions in the sequence simultaneously**, capturing long-range dependencies far better than RNNs or CNNs.
- Transformers remove the sequential bottleneck, enabling full parallelization during training and achieving superior performance on huge datasets.
- They form the backbone of modern NLP systems such as **BERT, GPT, T5**, and are rapidly expanding into vision, speech, and multimodal applications.

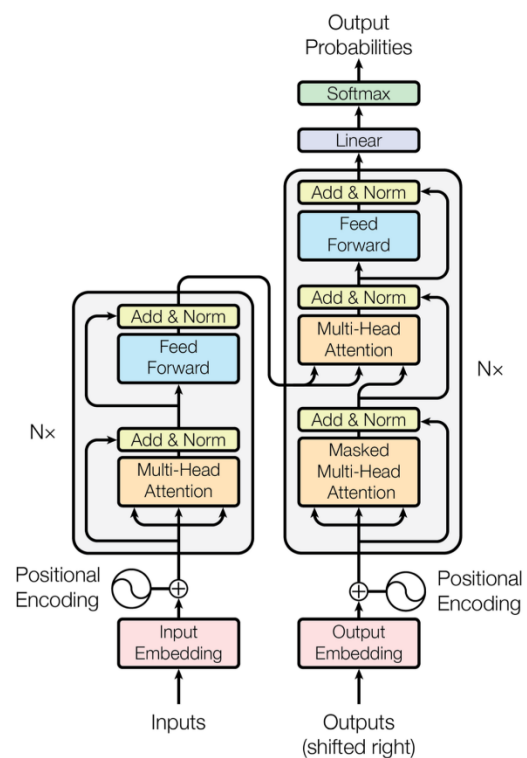


Figure 1: The Transformer - model architecture.

Feature Representation Learning

- Feature Representation Learning is the process by which a machine learning model **automatically discovers the best features** needed to solve a task, instead of depending on manually engineered features.
- It transforms raw data (like images, text, audio, or signals) into a more meaningful and useful internal representation that makes patterns easier to learn.
- The key idea is that the model learns **hierarchical features** — simple features in early layers and complex abstract features in deeper layers.

Why Is It Important?

- Traditional ML required heavy **manual feature engineering**, which needed expert domain knowledge and significant effort.
- Representation learning eliminates this requirement by letting the model extract its own features directly from raw input.
- This leads to better performance, especially on complex unstructured data like images or language.

How Representation Learning Works (Core Idea)

- The model receives raw input and passes it through multiple layers of transformations.
- Each layer learns to highlight certain patterns — for example:
 - In images: edges → textures → shapes → complete objects
 - In text: characters → words → phrases → semantic meaning
- The final learned representation captures the **essential structure** of the data and makes prediction easier for the classifier.

Examples of Representation Learning in Practice

1. **Convolutional Neural Networks (CNNs)**: learn hierarchical visual features such as edges, corners, textures, faces, objects.
2. **Recurrent Networks (RNN/LSTM/GRU)**: learn patterns in sequential data like grammar structures, speech rhythm, or time-dependent trends.
3. **Autoencoders**: compress data into a lower-dimensional latent vector that captures the most important information.
4. **Transformers**: learn contextual embeddings of words, where meaning is captured based on relationships across the entire sentence.
5. **Word Embeddings (Word2Vec, GloVe)**: represent words as dense vectors that encode semantic similarity.

Types of Feature Representation Learning

Type	Meaning	Example
Supervised Representation Learning	Learns features using labeled data	CNN for image classification
Unsupervised Representation Learning	Learns structure without labels	Autoencoders, PCA
Self-Supervised Learning	Creates its own labels from data	BERT masks words and predicts them
Transfer Learning	Uses features learned by one model for another task	Using pre-trained ResNet for new image task

Benefits of Representation Learning

- Automatically extracts relevant features
- Handles large-scale and complex data
- Reduces need for manual feature engineering
- Learns structure and patterns hidden in raw data
- Improves generalization and accuracy
- Forms the foundation of modern deep learning successes

Feature Engineering

Explain the Three Techniques Used Under Supervised Feature Selection

Supervised Feature Selection refers to methods that use the relationship between the features (\$X\$) and the target variable (\$Y\$) to determine which features are most useful. The three main techniques are Filter, Wrapper, and Embedded methods.

1. Filter Methods

- **Mechanism:** Features are selected based on their **statistical relevance** to the output variable, independent of any specific machine learning model.
- **Process:** Each feature is scored (or ranked) using a statistical measure. Features that score above a certain threshold are kept.
- **Examples:**
 - **Correlation Coefficient:** Measures linear relationship between a feature and the target variable.
 - **Chi-Squared Test (χ^2):** Measures the dependence between categorical features and a categorical target variable.
 - **ANOVA F-test:** Measures the difference in means of a numerical feature across different categories of the target variable.
- **Pros:** Fast, computationally inexpensive, and effective at eliminating obviously irrelevant features.

2. Wrapper Methods

- **Mechanism:** Features are selected by training and evaluating a specific machine learning model using subsets of features. The model acts as a "**wrapper**" around the feature selection process.
- **Process:** The model is trained repeatedly on different subsets. The best subset is chosen based on the model's performance metric (e.g., accuracy or F1-Score) measured via cross-validation.
- **Examples:**
 - **Forward Selection:** Starts with zero features and iteratively adds the feature that provides the greatest performance improvement.
 - **Backward Elimination:** Starts with all features and iteratively removes the least significant feature.
 - **Recursive Feature Elimination (RFE):** Selects features by recursively considering smaller and smaller sets of features.
- **Pros:** Highly accurate and finds the optimal feature subset for a specific model type.
- **Cons:** Computationally very expensive due to the repeated training and cross-validation cycles.

3. Embedded Methods

- **Mechanism:** Feature selection is performed **automatically during the model training process** (it's "embedded" within the algorithm).
- **Process:** The model itself is designed to assign weights or coefficients to features, and it penalizes features with little predictive power, effectively forcing their coefficients toward zero.
- **Examples:**
 - **LASSO Regression (L1 Regularization):** Adds a penalty equal to the absolute value of the magnitude of coefficients. This penalty can shrink the coefficients of irrelevant features exactly to zero, effectively selecting features.
 - **Tree-based Feature Importance:** Using the importance scores generated by Decision Trees, Random Forests, or Gradient Boosting Machines to select the top features.
- **Pros:** Combines the accuracy of Wrapper methods with the efficiency of Filter methods.

Explain the Benefits of Using Feature Selection in Machine Learning

Feature selection is a crucial preprocessing step that provides several significant benefits across model development and deployment:

1. **Reduced Overfitting:** By removing redundant and noisy features, the model's complexity is reduced, decreasing its **variance** and making it less likely to learn the noise in the training data (i.e., less prone to overfitting).
2. **Improved Model Interpretability:** Models trained on a smaller, more focused set of highly relevant features are much **easier to understand and explain** to stakeholders (e.g., "The model depends primarily on Age and Income, not shoe size").
3. **Faster Training and Inference:** Reducing the number of features shortens the training time because the algorithm has fewer parameters to optimize. It also speeds up prediction (**inference**) time in production.
4. **Reduced Data Collection Costs:** Identifying non-informative features means those data points do not need to be collected, stored, or processed in the future, saving computational and storage resources.
5. **Mitigation of the Curse of Dimensionality:** Feature selection directly addresses the "Curse of Dimensionality" by projecting the data into a lower, more manageable space.